

Universidad Tecnológica Nacional (UTN) - Facultad Regional San Nicolas

Carrera: Licenciatura en Programación

Trabajo Práctico Integrador

Asignatura: Arquitectura y Sistemas Operativos

Alumnos

Ivan Ezequias Cardozo - ivanezequiascardozoutn@gmail.com

Docente Titular

Martin Aristiaran

Docente Tutor

Santos Sanchez

25 de junio de 2026

ÍNDICE

Introducción	3
Marco Teórico	4
Caso Practico	5
Metodología Utilizada	6
Resultados Obtenidos	10
Conclusiones	13
Bibliografía	14
Anexo	15

INTRODUCCIÓN

El presente trabajo de investigación y aplicación práctica se centra en el estudio de la virtualización de recursos, una tecnología pilar en la administración moderna de infraestructuras informáticas. Se eligió este tema debido a su capacidad para demostrar, de manera tangible, cómo una capa de software es capaz de abstraer los recursos físicos de hardware para ejecutar múltiples sistemas operativos invitados de forma simultánea e independiente.

Para la formación integral de la carrera en Técnico Universitario en Programación, comprender la arquitectura detrás de la virtualización resulta fundamental. El dominio de esta herramienta permite la creación ágil de entornos de prueba aislados y controlados. Esto es crucial a la hora de depurar scripts, testear estructuras lógicas complejas o evaluar cómo interactúa el código con la asignación de memoria y el comportamiento del Kernel, garantizando que el desarrollo de software se realice de forma segura sin riesgo de comprometer el sistema operativo anfitrión ni el hardware físico.

Los objetivos principales que se proponen alcanzar con el desarrollo de este trabajo integrador son: en primer lugar, fundamentar teóricamente los conceptos de hipervisores y asignación de recursos; en segundo lugar, desplegar exitosamente una máquina virtual utilizando Oracle VirtualBox; y finalmente, instalar y configurar un servidor web Apache bajo una distribución Linux mínima, validando su correcta comunicación y funcionamiento dentro de una red local.

MARCO TEORICO

¿QUÉ ES LA VIRTUALIZACIÓN?

La virtualización es una tecnología que permite crear múltiples entornos simulados o recursos dedicados a partir de un único sistema de hardware. En lugar de destinar un servidor físico completo a una sola tarea o sistema operativo, la virtualización utiliza software para abstraer los componentes físicos (como el procesador, la memoria RAM y el almacenamiento) y dividirlos en varias máquinas virtuales (VM) que se encuentran aisladas entre sí. Cada una de estas máquinas virtuales funciona de manera independiente con su propio sistema operativo y aplicaciones, optimizando el uso de la infraestructura y reduciendo costos operativos.

HIPERVISOR (MONITOR DE MAQUINA VIRTUAL)

El núcleo de la virtualización es el hipervisor, también conocido como Monitor de Máquina Virtual (VMM). Se trata de una capa de software especializada que se encarga de crear, ejecutar y gestionar las máquinas virtuales. Su función principal es actuar como un mediador, separando el sistema operativo y las aplicaciones del hardware físico subyacente. El hipervisor asigna los recursos físicos a cada máquina virtual según sea necesario y garantiza que operen en entornos completamente aislados, evitando que un fallo en una VM afecte el resto del sistema.

CLASIFICACIÓN DE LOS HIPERVISORES

Los hipervisores se dividen en dos categorías principales dependiendo de su arquitectura y de cómo interactúan con el hardware físico:

- Hipervisor de Tipo 1 (Bare-Metal o Nativo): Este tipo de hipervisor se instala y ejecuta directamente sobre el hardware físico del servidor, sin necesidad de un sistema operativo anfitrión intermedio. Al tener acceso directo a los componentes de la máquina, ofrece un rendimiento superior, mayor estabilidad y alta seguridad. Son la opción estándar para entornos empresariales y centros de datos. Ejemplos de este tipo son VMware ESXi, Microsoft Hyper-V y Proxmox VE.
- Hipervisor de Tipo 2 (Alojado o Hosted): A diferencia del Tipo 1, este hipervisor se instala como una aplicación de software sobre un sistema operativo anfitrión ya existente (como Windows, macOS o Linux). Las peticiones de recursos de las máquinas virtuales deben pasar primero por el hipervisor y luego por el sistema operativo anfitrión antes de llegar al hardware. Aunque esto genera una ligera pérdida de rendimiento en comparación con el Tipo 1, es ideal para usuarios finales, entornos de desarrollo y pruebas de software. Un ejemplo clásico de este tipo es Oracle VM VirtualBox, el cual utilizamos para el desarrollo de la presente práctica.

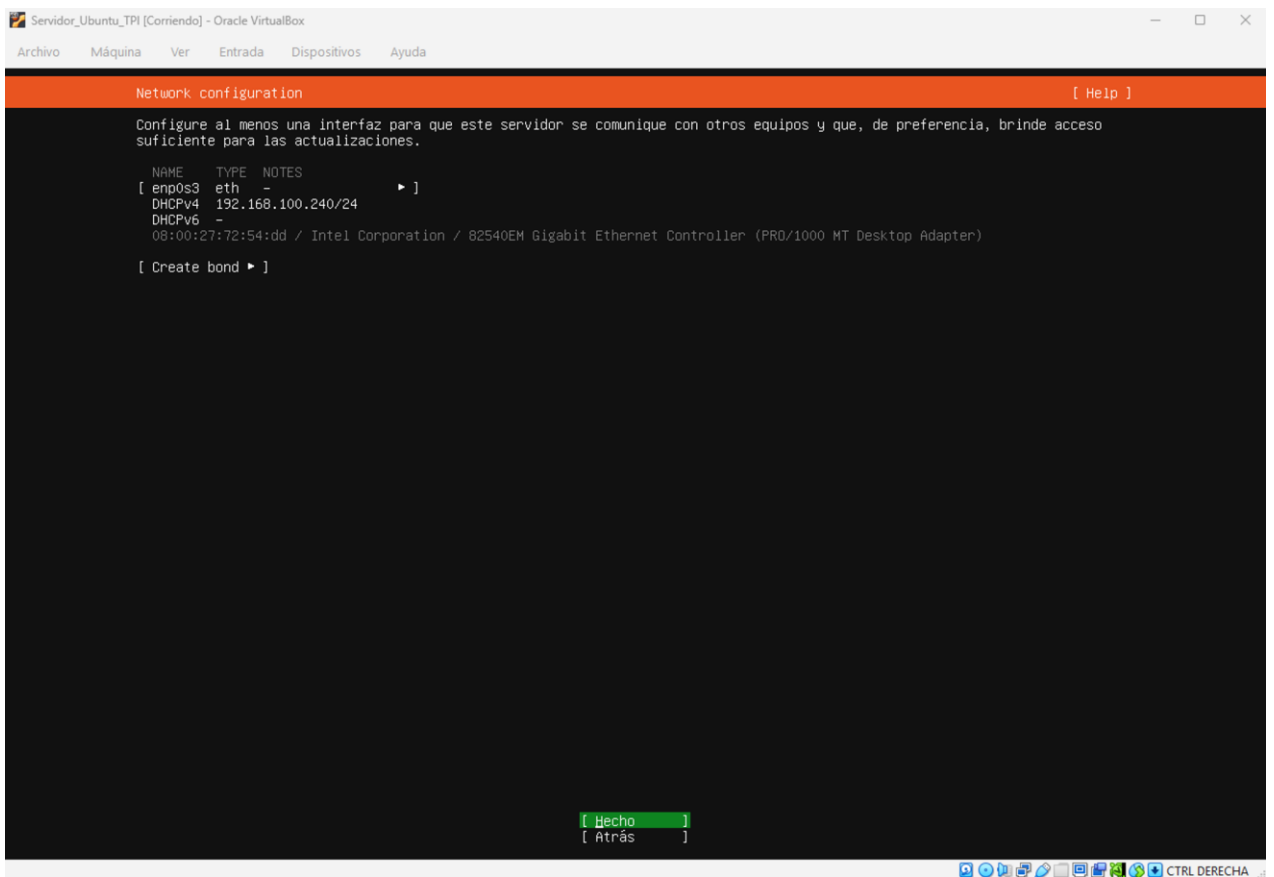
CASO PRÁCTICO

Para el desarrollo de esta práctica, se utilizó Oracle VM VirtualBox como hipervisor de Tipo 2. Se procedió a crear una nueva máquina virtual asignándole los recursos necesarios (procesador, memoria RAM y un disco duro virtual de 20 GB) para soportar un sistema operativo "Ubuntu Server (minimized)" sin comprometer el rendimiento del equipo físico. El objetivo central fue lograr que este servidor virtualizado alojara el servicio Apache y pudiera responder a peticiones externas desde la red local.

METODOLOGÍA UTILIZADA

DESCRIPCIÓN DEL ENTORNO Y CONFIGURACIÓN INICIAL

Este paso trata de la configuración de la red. Se seleccionó el modo "Adaptador Puente" (Bridged Networking), lo que permitió que la máquina virtual se conectara directamente al router local y obtuviera su propia dirección IP (192.168.100.240), haciéndola visible y accesible para otros dispositivos en la misma red.



```
Network configuration [ Help ]

Configure al menos una interfaz para que este servidor se comunice con otros equipos y que, de preferencia, brinde acceso
suficiente para las actualizaciones.

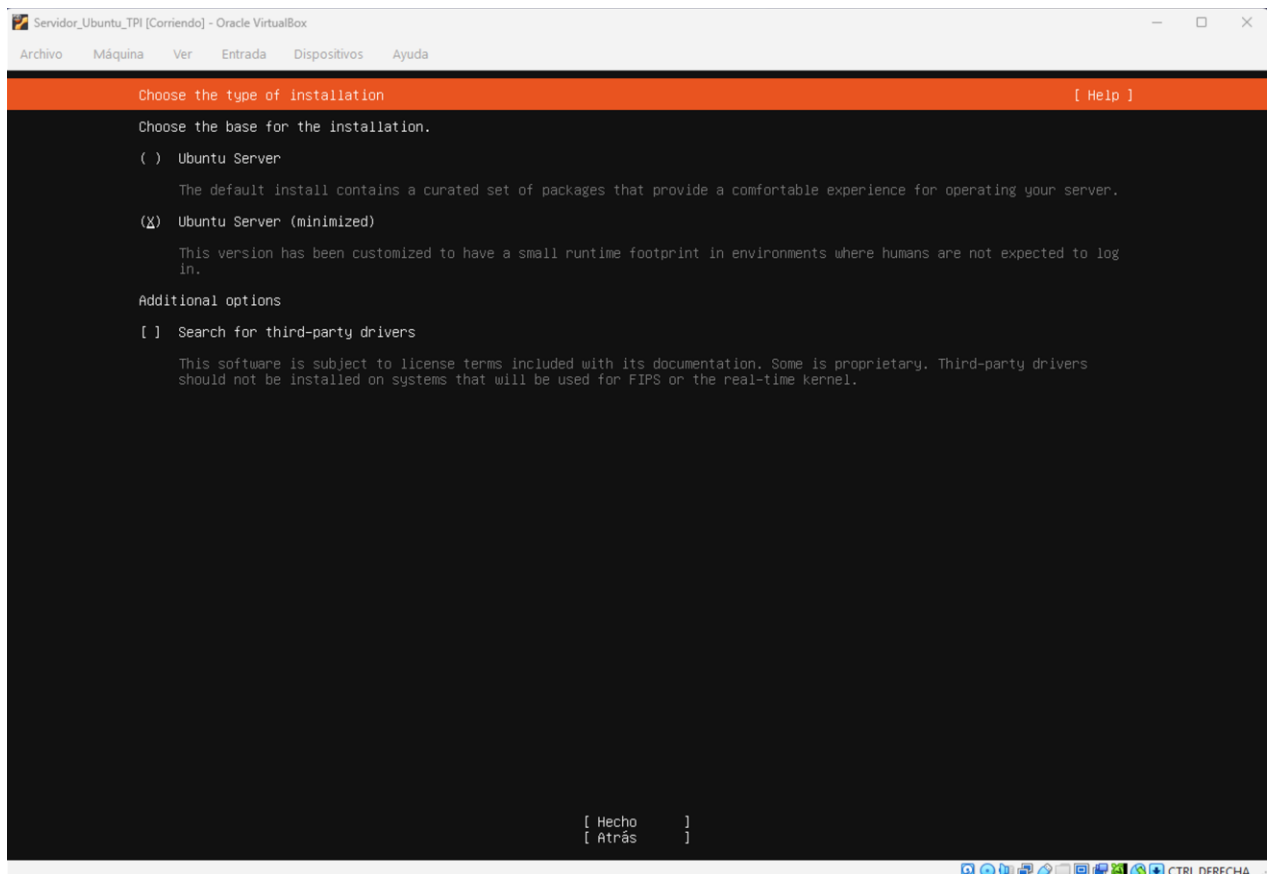
NAME    TYPE    NOTES
[ enp0s3 eth -           ► ]
DHCPv4  192.168.100.240/24
DHCPv6  -
08:00:27:72:54:dd / Intel Corporation / 82540EM Gigabit Ethernet Controller (PRO/1000 MT Desktop Adapter)

[ Create bond ► ]

[ Hecho ]
[ Atrás ]
```

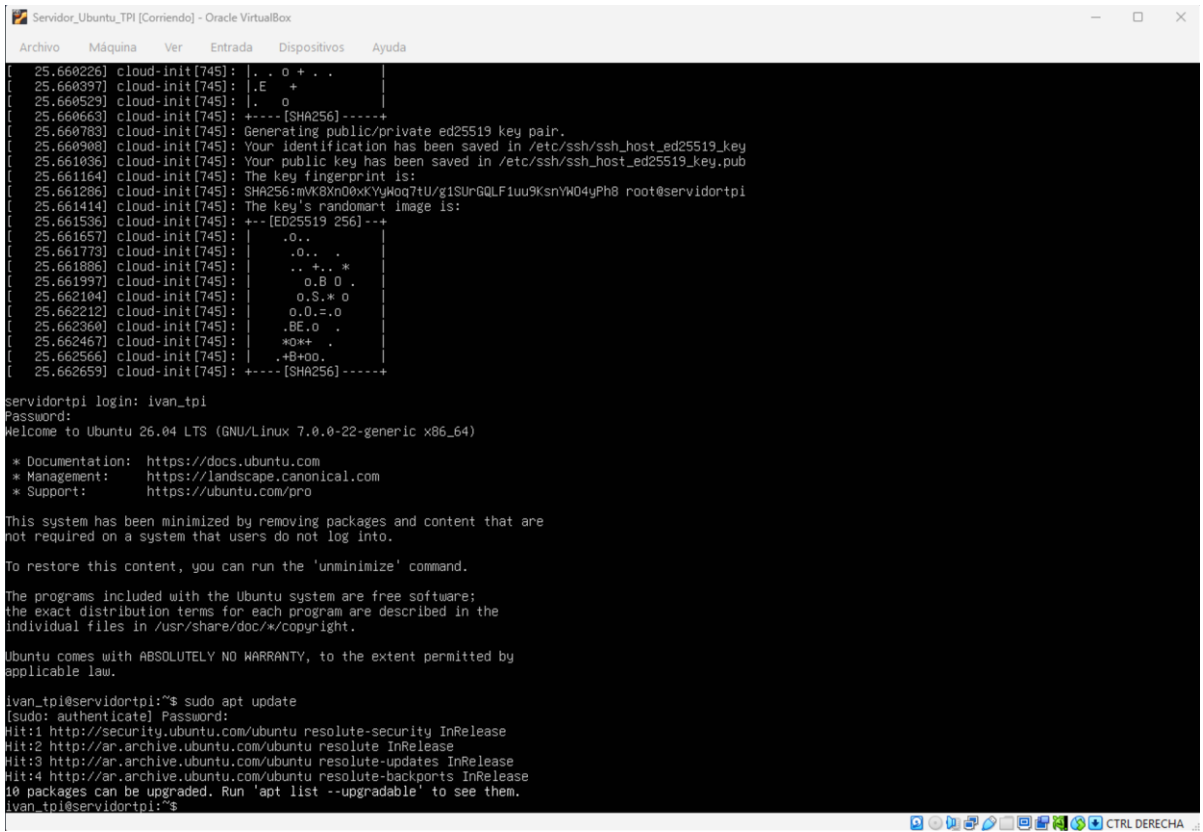
INSTALACIÓN DEL SISTEMA OPERATIVO

Se montó la imagen ISO de Ubuntu Server y se inició la máquina virtual. Siguiendo las directrices del trabajo, se optó por realizar una instalación mínima ("Ubuntu Server minimized"). Esta decisión técnica garantiza que el sistema ocupe el menor espacio posible en el disco y consuma menos recursos de hardware, al excluir paquetes y herramientas gráficas que no son necesarias para la administración mediante consola de comandos.



IMPLEMENTACIÓN DEL SERVIDOR WEB Y FIREWALL

Una vez finalizada la instalación del sistema operativo y tras el primer inicio de sesión exitoso con credenciales de administrador, se procedió a actualizar los repositorios locales (sudo apt update) e instalar el servidor web (sudo apt install apache2). Posteriormente, se configuró el cortafuegos para permitir el tráfico HTTP de entrada y salida, garantizando que el servidor pudiera responder a las peticiones externas.



```

Servidor_Ubuntu_TPI [Corriendo] - Oracle VM VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

[ 25.660226] cloud-init[745]: |. . o + . . |
[ 25.660397] cloud-init[745]: |.E  +      |
[ 25.660529] cloud-init[745]: |.  o      |
[ 25.660663] cloud-init[745]: +---- [SHA256] -----+
[ 25.660783] cloud-init[745]: Generating public/private ed25519 key pair.
[ 25.660908] cloud-init[745]: Your identification has been saved in /etc/ssh/ssh_host_ed25519_key
[ 25.661036] cloud-init[745]: Your public key has been saved in /etc/ssh/ssh_host_ed25519_key.pub
[ 25.661164] cloud-init[745]: The key fingerprint is:
[ 25.661286] cloud-init[745]: SHA256:mVKBXn00xYUHQq7tU/g1SurGQLF1uu9KsnYw04yPh8 root@servidortpi
[ 25.661414] cloud-init[745]: The key's randomart image is:
[ 25.661536] cloud-init[745]: +- [ED25519 256] +-+
[ 25.661657] cloud-init[745]: |.o..      |
[ 25.661773] cloud-init[745]: |.o..      |
[ 25.661886] cloud-init[745]: |..+..*    |
[ 25.661997] cloud-init[745]: |o.B 0 .    |
[ 25.662104] cloud-init[745]: |o.S.* o    |
[ 25.662212] cloud-init[745]: |o.O.=o    |
[ 25.662360] cloud-init[745]: |.BE.o .    |
[ 25.662467] cloud-init[745]: |*O*+ .    |
[ 25.662566] cloud-init[745]: |.+B+oo.   |
[ 25.662659] cloud-init[745]: +---- [SHA256] -----+

servidortpi login: ivan_tpi
Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ivan_tpi@servidortpi:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:2 http://ar.archive.ubuntu.com/ubuntu resolute InRelease
Hit:3 http://ar.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://ar.archive.ubuntu.com/ubuntu resolute-backports InRelease
10 packages can be upgraded. Run 'apt list --upgradable' to see them.
ivan_tpi@servidortpi:~$
```



```

Servidor_Ubuntu_TPI [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Setting up libapr1t64:amd64 (1.7.6-3) ...
Setting up liblua5.4-0:amd64 (5.4.8-1build1) ...
Setting up apache2-data (2.4.66-2ubuntu2.2) ...
Setting up libaprutil1t64:amd64 (1.6.3-3ubuntu3) ...
Setting up libaprutil1-dbd-sqlite3:amd64 (1.6.3-3ubuntu3) ...
Setting up apache2-utils (2.4.66-2ubuntu2.2) ...
Setting up apache2-bin (2.4.66-2ubuntu2.2) ...
Setting up apache2 (2.4.66-2ubuntu2.2) ...
Enabling module mpm_event.
Enabling module authz_core.
Enabling module authz_host.
Enabling module authn_core.
Enabling module auth_basic.
Enabling module access_compat.
Enabling module authn_file.
Enabling module authz_user.
Enabling module alias.
Enabling module dir.
Enabling module autoindex.
Enabling module env.
Enabling module mime.
Enabling module negotiation.
Enabling module setenvif.
Enabling module filter.
Enabling module deflate.
Enabling module status.
Enabling module reqtimeout.
Enabling conf charset.
Enabling conf localized-error-pages.
Enabling conf other-vhosts-access-log.
Enabling conf security.
Enabling conf serve-cgi-bin.
Enabling site 000-default.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache2.service' → '/usr/lib/systemd/system/apache2.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service' → '/usr/lib/systemd/system/apache-htcacheclean.service'.
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ivan.tpi@servidortpi:~$

```

```

Servidor_Ubuntu_TPI [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

Unpacking iptables (1.8.11-2ubuntu3) ...
Selecting previously unselected package libnftables1:amd64.
Preparing to unpack .../6-libnftables1_1.1.6-1_amd64.deb ...
Unpacking libnftables1:amd64 (1.1.6-1) ...
Selecting previously unselected package nftables.
Preparing to unpack .../7-nftables_1.1.6-1_amd64.deb ...
Unpacking nftables (1.1.6-1) ...
Selecting previously unselected package ufw.
Preparing to unpack .../8-ufw_0.36.2-9build1_all.deb ...
Unpacking ufw (0.36.2-9build1) ...
Setting up libip4tc2:amd64 (1.8.11-2ubuntu3) ...
Setting up libip6tc2:amd64 (1.8.11-2ubuntu3) ...
Setting up libnftnl1:amd64 (1.3.1-1) ...
Setting up libnftnlk0:amd64 (1.0.2-3build1) ...
Setting up libnftables1:amd64 (1.1.6-1) ...
Setting up nftables (1.1.6-1) ...
Setting up libnetfilter-conntrack3:amd64 (1.1.1-1) ...
Setting up iptables (1.8.11-2ubuntu3) ...
update-alternatives: using /usr/sbin/iptables-legacy to provide /usr/sbin/iptables (iptables) in auto mode
update-alternatives: using /usr/sbin/iptables-legacy to provide /usr/sbin/iptables (ip6tables) in auto mode
update-alternatives: using /usr/sbin/iptables-nft to provide /usr/sbin/iptables (iptables) in auto mode
update-alternatives: using /usr/sbin/iptables-nft to provide /usr/sbin/iptables (ip6tables) in auto mode
update-alternatives: using /usr/sbin/arptables-nft to provide /usr/sbin/arptables (arptables) in auto mode
update-alternatives: using /usr/sbin/ebtables-nft to provide /usr/sbin/ebtables (ebtables) in auto mode
Setting up ufw (0.36.2-9build1) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline
creating config file /etc/ufw/before.rules with new version
creating config file /etc/ufw/after.rules with new version
creating config file /etc/ufw/after6.rules with new version
Created symlink '/etc/systemd/system/multi-user.target.wants/ufw.service' → '/usr/lib/systemd/system/ufw.service'.
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ivan.tpi@servidortpi:~$ sudo ufw allow 'Apache'
Rules updated
Rules updated (v6)
ivan.tpi@servidortpi:~$

```

RESULTADOS OBTENIDOS

Al ingresar la dirección IP de la máquina virtual (192.168.100.240) en un navegador web estándar desde el sistema operativo anfitrión (Windows), se logró visualizar exitosamente la página de bienvenida "Apache2 Default Page". Esto confirmó que el servicio web estaba corriendo adecuadamente y que el aislamiento de red funcionaba según lo previsto.



Apache2 Default Page

Ubuntu **It works!**

This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented** in `/usr/share/doc/apache2/README.Debian.gz`. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

```
/etc/apache2/
|-- apache2.conf
|   |-- ports.conf
|   |-- mod-enabled
|   |   |-- *.load
|   |   |-- *.conf
|   |-- conf-enabled
|   |   |-- *.conf
|   |-- sites-enabled
|   |   |-- *.conf
```

- `apache2.conf` is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
- `ports.conf` is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
- Configuration files in the `mod-enabled/`, `conf-enabled/` and `sites-enabled/` directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
- They are activated by symlinking available configuration files from their respective `*-available/` counterparts. These should be managed by using our helpers `a2enmod`, `a2dismod`, `a2ensite`, `a2dissite`, and `a2enconf`, `a2disconf`. See their respective man pages for detailed information.
- The binary is called `apache2` and is managed using `systemd`, so to start/stop the service use `systemctl start apache2` and `systemctl stop apache2`, and use `systemctl status apache2` and `journalctl -u apache2` to check status. `systemd` and `apache2ctl` can also be used for service management if desired. **Calling `/usr/bin/apache2` directly will not work** with the default configuration.

Document Roots

By default, Ubuntu does not allow access through the web browser to any file apart from those located in `/var/www/public_html` directories (when enabled) and `/usr/share` (for web applications). If your site is using a web document root located elsewhere (such as in `/srv`) you may need to whitelist your document root directory in `/etc/apache2/apache2.conf`.

The default Ubuntu document root is `/var/www/html`. You can make your own virtual hosts under `/var/www`.

Reporting Problems

Please use the `ubuntu-bug` tool to report bugs in the Apache2 package with Ubuntu. However, check **existing bug reports** before reporting a new bug.

Please report bugs specific to modules (such as PHP and others) to their respective packages, not to the

Como resultado culminante de la configuración del servidor, se logró validar la operatividad de un entorno de desarrollo completamente funcional y aislado. Para evidenciar este logro, se redactó y ejecutó un script de prueba en Python (`promedios.py`) operando estrictamente desde la interfaz de línea de comandos mediante el editor nano.

La ejecución de este código demuestra los siguientes resultados operativos sobre el sistema invitado:

- Estabilidad del Entorno: Se comprobó que la máquina virtual con Ubuntu Server, en su versión minimizada, es capaz de interpretar y ejecutar sintaxis de Python de manera nativa, sin requerir una interfaz gráfica de usuario (GUI).
- Procesamiento Lógico y de Memoria: El sistema procesó correctamente las estructuras de control de flujo (bucle `for`) y la gestión de memoria dinámica, almacenando los ingresos del usuario en una lista mediante el método `.append()`.
- Precisión Matemática: Las operaciones aritméticas delegadas a las funciones integradas del núcleo (`sum()` y `len()`) calcularon el promedio exacto utilizando variables de punto flotante (`float`).
- Salida Formateada: El servidor respondió exitosamente a las directivas de formato de cadena (f-strings), restringiendo la salida a dos decimales (`:.2f`) para entregar un resultado limpio en consola.

Con esta demostración práctica, se da por cumplido el objetivo principal del laboratorio: disponer de un servidor virtualizado, seguro y estable, ideal para la realización de pruebas de software sin comprometer los recursos ni la integridad del sistema operativo anfitrión.

```
GNU nano 8.7.1 promedios.py
print ("Calculadora de Promedios")
notas = []
for i in range (3):
    nota = float(input(f"Ingrese la nota {i + 1}: "))
    notas.append(nota)
print (f"El promedio final es: {sum(notas)/len(notas):.2f}")

ivan_toi@servidor01:~$
```

CONCLUSIONES

Durante la configuración del cortafuegos, el sistema arrojó el error “ufw: command not found”. Esto ocurrió porque que la versión minimizada de Ubuntu excluye este paquete por defecto. El inconveniente se resolvió ejecutando la instalación manual del firewall (sudo apt install ufw), lo que permitió aplicar las reglas de acceso web sin mayores contratiempos.



```
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ivan_tp1@servidortp1:~$ sudo ufw allow 'apache'
sudo: 'ufw': command not found
ivan_tp1@servidortp1:~$ sudo ufw allow 'Apache'
```

La realización de este trabajo integrador permitió consolidar de forma práctica la teoría sobre virtualización. Se demostró la eficacia de los hipervisores para desplegar servicios de red de manera rápida y segura sin alterar el hardware anfitrión.

Más allá del despliegue del servidor web Apache, este trabajo comprobó la utilidad de los entornos virtualizados como herramienta indispensable en el ciclo de desarrollo de software. Contar con un sistema operativo aislado permite realizar pruebas de código, analizar de cerca el comportamiento del kernel y evaluar cómo interactúan los programas con la asignación de memoria en un entorno completamente controlado. De esta forma, es posible validar el funcionamiento de aplicaciones y servicios mitigando cualquier riesgo de comprometer la estabilidad del sistema o la integridad de la máquina física principal.

BIBLIOGRAFÍA

Material teórico por la catedra:

- [Virtualización: Concepto y Beneficios](#)
- [Hipervisor Tipo 1 y Tipo 2](#)
- [Virtualización de recursos](#)

Silberschatz, A., Galvin, P. B., Gagne, G., Wiley, 2018. [Operating System Concepts](#)

Tanenbaum, A. S., Bos, H, Pearson, 2021. [Modern Operating Systems](#)

Oracle VM VirtualBox. [Documentación oficial](#)

Ubuntu server [Documentación oficial](#)

Apache2 [Documentación oficial](#)

ANEXO

Se adjunta un repositorio en la nube con las capturas de pantalla originales en alta resolución para su correcta visualización y auditoría. Así como también los enlaces correspondientes al video demostrativo y al repositorio:

[Link Fotos \(Google Drive\)](#)

[Link Repositorio \(GitHub\)](#)

[Link Video \(YouTube\)](#)